UNIVERSITAT JAUME I (UJI) · EI-1057 MOBILE NETWORKS & DEVICES

100% ENGLISH ORAL DEFENSE

SimmerLink: Connecting the Dots Across the Mobile Network Stack

An Offline-First Android & iOS Culinary Companion Integrating **BLE 5.0 GATT Sensors**, **MANET UDP Discovery**, **Full-Duplex WebSocket P2P**, and **Multimodal Cloud AI**.

DOT 1 · OFFLINE & OS

SQLite + Native Radio

Works at **0 bps** in supermarket basements via `Drift` + Kotlin `ConnectivityManager`, `WifiManager` & iOS `getifaddrs`.

DOT 2 · MANET DISCOVERY

UDP Broadcast :41234

Zero-configuration peer discovery via `RawDatagramSocket` subnet beacons every 4 seconds.

DOT 3 · WLAN CO-OP

Full-Duplex WS :8765

Sub-10ms real-time shopping checkoffs & Cook Mode step sync with `_pkt_id` loop prevention.

DOT 4 · WPAN & WWAN

BLE 5.0 GATT + Gemini AI

Binary `0x181A` Thermometer & `0x181D` Scale codecs + ESP32 C++ firmware & Gemini 2.5 Flash vision.

Presentation Roadmap (Signposting): 1. Offline & OS Lifecycle → 2. 5-Layer Stack → 3. Native Radio API → 4. UDP vs. TCP P2P → 5. BLE Binary Framing → 6. Visual Demo & Verification

120 / 120 TESTS PASSING

OFFLINE-FIRST CORE

Why Cloud-Only Kitchen Apps Fail on Mobile Devices

Three physical kitchen & supermarket constraints solved through clean mobile systems engineering:

1. Supermarket Dead Zones (0 bps)

- **Problem:** Basement supermarkets lose LTE/5G signal right when users need their grocery list.
- **Solution:** Reactive relational **SQLite (`Drift`)** in `app_database.dart` (`Recipes` , `Ingredients` , `ShoppingListItems`).
- **Impact:** Deterministic ingredient aggregation (`SmartShoppingAggregator`) with zero network latency.

2. OS Background Doze & Lock

- **Problem:** Android/iOS throttles `Timer.periodic` when the phone screen locks or backgrounds.
- **Solution:** Wall-clock `targetEndTime` timestamp + `WidgetsBindingObserver` (`AppLifecycleState.resumed`).
- **Impact:** Zero timer drift upon unlock + hands-free display wake via `WakelockPlus` .

3. Multi-Device Kitchen Latency

- **Problem:** Two people shopping or cooking together need instant state sync without cloud logins.
- **Solution:** Local ad-hoc **UDP :41234 discovery** + **WebSocket :8765** P2P sync.
- **Impact:** **< 10 ms** LAN latency across Android & iOS devices.

5 WIRELESS LAYERS

Five Networking Layers Orchestrated in One App

Selecting the optimal transport protocol for each physical range, power budget, and reliability requirement:

EI-1057 NETWORK LAYER	PROTOCOL / PORT / UUID	ENGINEERING PURPOSE IN SIMMERLINK	CODE IMPLEMENTATION
Native OS Radio API	`MethodChannel("com.mobkitchen/native_network")`	Reads physical RSSI (dBm), Link Speed (Mbps), 2.4/5 GHz band, `MulticastLock` & BLE state	`MainActivity.kt` & `AppDelegate.swift`
WPAN (Short-Range IoT)	**BLE 5.0 GATT** (`0x181A` / `0x181D`)	Binary core meat temperature (`0x2A6E` , 2B) & 0.1g load-cell scale weight (`0x2A9D` , 3B)	`lan_sync_engine.dart` & `esp32_smart_kitchen_node.ino`
MANET (Ad-Hoc Discovery)	**UDP Broadcast** (`255.255.255.255:41234`)	Zero-config peer discovery (`simmer_beacon` JSON) every 4s on local `/24` subnet	`LanSyncAndBleCodecEngine.startUdpDiscovery()`
WLAN (Local P2P Sync)	**TCP HTTP + WebSocket** (`Port 8765`)	Full-duplex Co-Op supermarket checkoffs & active cooking step synchronization	`startServer()` & `connectToPeerWebSocket()`
WWAN (Cloud AI)	**HTTPS REST / TLS** (`gemini-2.5-flash`)	Multimodal fridge photo ingredient vision, URL recipe import & AI Sous-Chef	`GeminiRecipeServiceImpl` (`lib/features/ai/`)

KOTLIN + SWIFT BRIDGE

Android Kotlin & iOS Swift Radio Diagnostics Bridge

Bridging OS-level hardware radio telemetry into Flutter via `MethodChannel("com.mobkitchen/native_network")` :

🤖 Android Native (`MainActivity.kt`)

- **`ConnectivityManager` + `NetworkCapabilities` :** Detects active transport (`TRANSPORT_WIFI` , `TRANSPORT_CELLULAR` , `TRANSPORT_ETHERNET`) and Android 10+ `caps.signalStrength` .
- **`WifiManager.connectionInfo` :** Reads physical **RSSI (dBm)**, **Link Speed (Mbps)**, and **Frequency (MHz)** (classifying `≥ 4900 MHz` as **5 GHz** vs. **2.4 GHz**).
- **`LinkProperties` & `NetworkInterface` :** Prioritizes private RFC-1918 LAN IPv4 on `wlan0` / `ap0` (`192.168.x.x`).
- **`BluetoothManager.adapter` :** Reports hardware BLE radio state (`ON` / `OFF`).

🔒 Android 12–15 Security & iOS `getifaddrs`

- **`AndroidManifest.xml` (API 31–36):** `NEARBY_WIFI_DEVICES` & `BLUETOOTH_SCAN` with `android:usesPermissionFlags="neverForLocation"` — discovers BLE & Wi-Fi peers without harvesting user GPS location.
- **`CHANGE_WIFI_MULTICAST_STATE` + `WifiManager.MulticastLock` :** Acquires `simmerlink_udp_manet_lock` in `MainActivity.kt` so Android Wi-Fi chipsets do not filter incoming UDP Broadcast (`:41234`) packets.
- **iOS Native (`AppDelegate.swift`):** Inspects BSD POSIX `getifaddrs` on interface `en0` (`AF_INET`) to resolve Wi-Fi IPv4 natively on iPhone & iOS Simulator.

UDP :41234 + WS :8765

Why UDP Broadcast (:41234) + Full-Duplex WebSocket (:8765)?

Two-phase local peer-to-peer protocol in `lib/features/wireless/data/lan_sync_engine.dart`:

PHASE 1 · CONNECTIONLESS DISCOVERY

UDP Broadcast (`RawDatagramSocket` :41234)

- Binds to `0.0.0.0:41234` with `reuseAddress = true` and `broadcastEnabled = true`.
- Broadcasts every **4 seconds** to `255.255.255.255:41234` + directed `/24` subnet broadcast (`192.168.x.255`): `{"type":"simmer_beacon","ip":"192.168.1.42","port":8765,"room":"SIMMER-2026"}`
- **Why not TCP scan?** Scanning 254 IPs via TCP requires 254 3-way handshakes and triggers OS firewall rate limits; 1 UDP datagram reaches the subnet in **< 2 ms**.

PHASE 2 · RELIABLE FULL-DUPLEX SYNC

TCP HTTP + WebSocket (`HttpServer` :8765)

- Upgrades discovered peers automatically to `ws://<peer-ip>:8765/ws` (with `POST /sync` HTTP fallback).
- **Echo-Storm Prevention (`_pkt_id`):** Every frame carries a unique ID checked against a 256-entry `_seenPacketIds` ring buffer so mutually connected phones never loop packets infinitely.
- **Socket Reuse (`_outboundSocketsByEndpoint`):** Reuses open WebSockets across periodic 4-second UDP beacons.

BLE 5.0 GATT BINARY

BLE 5.0 GATT Binary Codecs + ESP32 Firmware & Physics Engine

Fitting within the 20-byte BLE ATT payload MTU with 90% less radio airtime than JSON:

🌡️ Service `0x181A` · Temperature (`0x2A6E`)

Format: 2-Byte Signed `int16` Little-Endian (**0.01 °C** resolution)

GATT 0x2A6E Little-Endian Frame (56.00 °C Target Steak): `56,00`

0xE0	0x15	5600
LSB (Byte 0)	MSB (Byte 1)	Raw int16

Models **Newton's Law of Heating**: $T(t) = T_{env} + (T_0 - T_{env})\cdot e^{-kt}$ with live ETA countdown & negative freezer support (`−18.50°C`).

⚖️ Service `0x181D` · Smart Scale (`0x2A9D`)

Format: 1-Byte Flags (`0x00` SI) + 2-Byte `uint16` Little-Endian (**0.1 g** resolution)

GATT 0x2A9D Little-Endian Frame (125.4 g Pour): `125,4`

0x00	0xE6	0x04
Flags (SI)	LSB (Byte 1)	MSB (Byte 2)

Matches `iot_firmware/esp32_smart_kitchen_node.ino` (C++ `BLE2902` notifications + Wi-Fi HTTP `POST /telemetry`).

APP-DEBUG.APK (SDK 36) · 0 ANALYZER ISSUES · 120 / 120 TESTS PASSING

Live Wireless Hub Walkthrough: Device Radar · BLE Telemetry · Wi-Fi Co-Op

1 · Cihaz Radarı (Device Radar)

● 2 BLE Aktif

Cihaz Radarı | Canlı Telemetri | Wi-Fi Co-Op

SimmerLink Probe Pro (BLE 0x181A) — Bağlı ✓

GATT 0x2A6E · AA:BB:CC:18:1A:01

BLE 5.0 GATT · -52 dBm (80%) · 94%

MobScale Precision (0x181D) — Bağlı ✓

GATT 0x2A9D · 0.1g Load-Cell

BLE 5.0 GATT · -58 dBm (70%) · 87%

Native OS Network Diagnostics — Sorgula ↻

MethodChannel: `com.mobkitchen/native_network`

Taşıyıcı: WIFI · RSSI: -52 dBm · 866 Mbps (5 GHz)

IPv4: 192.168.1.42 · BT: ON · MulticastLock

2 · Canlı Telemetri (BLE GATT)

● CANLI AKIŞ

Cihaz Radarı | Canlı Telemetri | Wi-Fi Co-Op

BLE Et & Tava Termometresi — 0x2A6E [0x31, 0x15]

ÇEKİRDEK SICAKLIK: **54.2°C** (+14.5 °C/dk hız)

HEDEF (STEAK): **56°C** (Tahmini: 0dk 08sn)



Orta-Az Steak (56°C) · Isıyı Artır (+7.5°C)

BLE Akıllı Tartı (0x181D) — Dara Al (Tare)

Hedef: Makarna (400 g) · ✅ Tam Gramaj! (0x00 0xA0 0x0F)

400.0 g

3 · Wi-Fi Co-Op (UDP + WS P2P)

● SUNUCU AKTİF

Cihaz Radarı | Canlı Telemetri | Wi-Fi Co-Op

Wi-Fi LAN Co-Op Sunucusu — :8765 + :41234

Aktif: `ws://192.168.1.42:8765/ws` · Oda: `SIMMER-2026`

UDP MANET (:41234): AKTİF · Bağlı Eşler: 1

Adım: spicy_vodka_rigatoni:2/4

Co-Op Canlı Senkron Aksiyonları

Market İşaretlemesi Simüle Et

Simulate Peer Cook Step Sync

Canlı Paket Günlüğü (Packet Log)

⇄ Pixel 9 Pro (:41234): UDP `simmer_beacon` keşfedildi → WS bağlandı.

⇄ Pixel 9 Pro (WS :8765): Cook Mode Sync: Step 2/4 (`_pkt_id` verified).

⇄ Sous-Chef iPhone: "Sarımsak" SQLite market listesinde işaretlendi!

THANK YOU · Q&A

"Stay Hungry, Stay Foolish" — Connecting Every Dot

Full alignment with the EI-1057 curriculum and UJI English Oral Presentation Rubric:

✅ UJI Oral Presentation Rubric Coverage

- **Storytelling & Hook (Steve Jobs Framework):** Relatable kitchen & basement dead-zone story → 4 connected networking dots.
- **Signposting & Structure:** Explicit transitions across Problem → Stack → Native API → UDP/TCP → BLE → Visual Demo.
- **EI-1057 Technical Rigor:** Real sockets (`RawDatagramSocket` , `HttpServer` , `WebSocket`), binary `ByteData` codecs, and Kotlin/Swift `MethodChannel` .
- **Visual Do's & Don'ts:** High-contrast visual cards, byte frame diagrams, 3-tab mobile UI walkthrough, zero walls of text.

💬 Ready for Jury Questions (Q&A Highlights)

- **Why UDP :41234 + TCP :8765?** 1 UDP subnet broadcast discovers peers in <2ms without 254 TCP handshakes; WebSocket guarantees ordered sync.
- **How do you prevent WebSocket echo storms?** Bounded 256-entry `_seenPacketIds` ring buffer + `_outboundSocketsByEndpoint` socket reuse.
- **Why binary GATT `0x2A6E` / `0x2A9D` instead of JSON?** Fits inside the 20-byte BLE ATT MTU (2–3 bytes vs. 35+ bytes JSON), saving 90% radio airtime.